Airspace Intelligence

Real-Time Aircraft Monitoring over the Iberian Peninsula

Modern Data Architectures for Big Data II — IE University

Group 4 Members

#	Name
1	Rincon Juan José
2	Gelin Romain
3	Gaitan Diego
4	Tcheishvili Luka
5	Tambey Cécile

Table of Contents

---

1. Project Overview

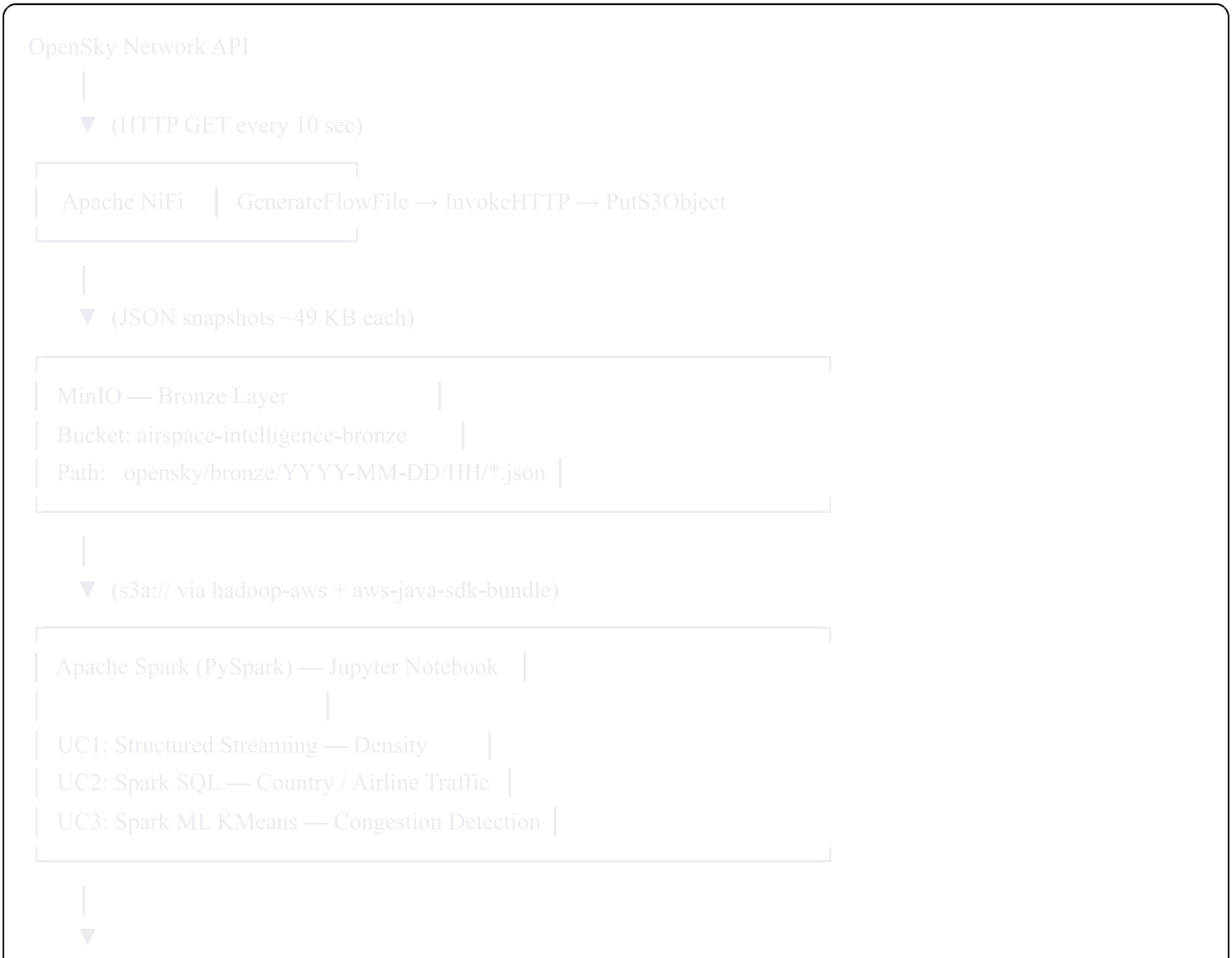
Airspace Intelligence is a real-time Big Data pipeline that continuously ingests live aircraft telemetry from the

OpenSky Network API, stores raw JSON snapshots in **MinIO** object storage, and performs three analytical use cases using **Apache Spark**.

Field	Detail
Monitored Region	Iberian Peninsula (lat 35.5–44.5°N, lon -10.0–4.5°E)
Ingestion Frequency	Every 10 seconds
Data Source	OpenSky Network REST API
Spark Features Used	Structured Streaming · SQL · ML (KMeans)
Submission Deadline	March 19th, 2026 via Blackboard

---

2. Architecture



Visualisations
Seaborn · Matplotlib
Folium (HTML map)

---

3. Access Credentials *(Confidential)*

> **For team members and professor only. Do not share publicly.**

Credential	Value
MinIO Access Key	`airspace-intel`
MinIO Secret Key	`Airspace2026Stream!`
MinIO S3 Endpoint	`http://127.0.0.1:9000`
MinIO Web Console	`http://127.0.0.1:9001`
Bronze Bucket	`airspace-intelligence-bronze`
Spark Bronze Path	`s3a://airspace-intelligence-bronze/opensky/bronze/`
OpenSky API	`https://opensky-network.org/api/states/all`
Bounding Box	`lamin=35.5 lomin=-10.0 lamax=44.5 lomax=4.5`

---

4. MinIO Setup

4.1 Start the Server

```bash
minio server --address :9000 --console-address :9001 /data/s3
```

Flag	Purpose
`--address :9000`	S3 API port — used by NiFi and Spark
`--console-address :9001`	Web UI port — open in browser to manage buckets
`/data/s3`	Physical directory where all bucket data is stored

4.2 Verify

```bash
sudo ss -tulnp | grep 9000   # confirm port is listening
sudo service minio status    # confirm service is active
```

Then open `http://127.0.0.1:9001` in a browser and log in with the credentials above.

4.3 Create the Bronze Bucket

1. In the MinIO console, click **Buckets** in the left sidebar
2. Click **Create Bucket**
3. Enter name exactly: `airspace-intelligence-bronze`
4. Leave all other settings as default — Access will show as **PRIVATE** (this is correct)
5. Click **Create Bucket**

After ingestion begins, bucket details will match:

Field	Expected Value
Bucket Name	`airspace-intelligence-bronze`
Access	`PRIVATE`
File path	`opensky/bronze/YYYY-MM-DD/HH/opensky_YYYYMMDD_HHmmss_SSS.json`
File size per snapshot	~49.2 – 50.0 KiB
Total after 67 snapshots	~3.1 MiB · 67 objects

4.4 Validate Data Arrival (after NiFi is running)

Navigate to `airspace-intelligence-bronze/opensky/bronze/YYYY-MM-DD/HH/` — you should see `.json` files (~49–

50 KB each) named like:

```
opensky_20260312_183050_123.json
```

> **Note:** Files are stored in dated subfolders. This is why Spark requires `recursiveFileLookup=true` when reading.

---

5. Apache NiFi Pipeline

Flow order: `GenerateFlowFile → InvokeHTTP → LogAttribute → PutS3Object`

> Start MinIO **before** starting NiFi. There are **4 processors** in the pipeline.

5.1 Create Process Group

1. Open NiFi UI (typically `http://127.0.0.1:8080/nifi`)
2. Drag a **Process Group** onto the canvas
3. Name it: `airspace-intelligence`
4. Double-click to enter the group

5.2 Processor 1 — GenerateFlowFile

Triggers the pipeline on a 10-second schedule by producing an empty FlowFile.

SCHEDULING tab:

Setting	Value
Run Schedule	`10 sec`

PROPERTIES tab:

Property	Value
File Size	`0 b`
Batch Size	`1`

5.3 Processor 2 — InvokeHTTP

Makes the HTTP GET request to the OpenSky API and returns the JSON as FlowFile content.

PROPERTIES tab:

Property	Value
HTTP Method	GET
Remote URL	https://opensky-network.org/api/states/all?lamin=35.5&lomin=-10.0&lamax=44.5&lomax=4.5
Connection Timeout	10 sec
Read Timeout	30 sec
Follow Redirects	True

RELATIONSHIPS tab: Auto-terminate Failure, No Retry, Original, Retry. Connect only Response to the next processor.

5.4 Processor 3 — PutS3Object

Writes the FlowFile to the MinIO Bronze bucket using the S3 protocol.

PROPERTIES tab:

Property	Value
Bucket	`airspace-intelligence-bronze`
Object Key	`opensky/bronze/${now():format('yyyy-MM-dd')}/${now():format('HH')}/opensky_${now():format('yyyyMMdd_HHmmss_SSS')}.json`
Region	`us-east-1`
Access Key ID	`airspace-intel`
Secret Access Key	`Airspace2026Stream!`
Endpoint Override URL	`http://127.0.0.1:9000`
Use Path Style Access	`True`
SSL Context Service	*(leave empty — HTTP not HTTPS)*

RELATIONSHIPS tab: Auto-terminate `Failure` and `Success`.

> The Object Key uses NiFi Expression Language to auto-create date/hour folder hierarchy.

5.5 Processor 3 — LogAttribute

Logs FlowFile attributes between InvokeHTTP and PutS3Object for debugging and monitoring.

PROPERTIES tab:

Property	Value
Log Level	`info`
Log Payload	`false`

RELATIONSHIPS: Connect `success` to PutS3Object.

5.6 Connect and Start

1. **GenerateFlowFile** → (success) → **InvokeHTTP**
2. **InvokeHTTP** → (Response) → **LogAttribute**
3. **LogAttribute** → (success) → **PutS3Object**
4. Auto-terminate all unused relationships on each processor
5. Right-click canvas → **Select All** → click **Start**
6. All processors should turn green
7. After 60 seconds, verify files appear in MinIO console

---

6. Jupyter Notebook — Spark Analysis

6.1 Key Spark Configuration

```python
spark = (SparkSession.builder
    .appName("AirspaceIntelligence")
    .config("spark.jars.packages",
            "org.apache.hadoop:hadoop-aws:3.3.4,"
            "com.amazonaws:aws-java-sdk-bundle:1.12.262")
    .config("spark.hadoop.fs.s3a.endpoint",       "http://127.0.0.1:9000")
    .config("spark.hadoop.fs.s3a.access.key",     "airspace-intel")
    .config("spark.hadoop.fs.s3a.secret.key",     "Airspace2026Stream!")
    .config("spark.hadoop.fs.s3a.path.style.access", "true")
    .config("spark.hadoop.fs.s3a.connection.ssl.enabled", "false")
    .getOrCreate())
```

6.2 Critical Data Load Pattern

```python
raw_df = (
    spark.read
    .schema(opensky_schema)
    .option("multiLine", True)
    .option("recursiveFileLookup", "true")   # REQUIRED — files are in subfolders
    .json(BRONZE_PATH)
)
```

6.3 Use Case 1 — Structured Streaming (Density Monitoring)

- Reads Bronze path as a **stream**
- Applies a **1-minute tumbling window** on snapshot timestamp
- Uses `approx_count_distinct` (not `countDistinct` — exact distinct is unsupported in streaming)
- Sinks to memory table `density_stream`, runs for 25 seconds

6.4 Use Case 2 — Spark SQL (Country & Airline Traffic)

- Registers DataFrame as temp view: `aircraft_df.createOrReplaceTempView('aircraft')`
- SQL query for top-20 countries by unique aircraft
- Airline code extracted with regex: first 3 uppercase alpha characters of callsign
- Speed converted: `avg_velocity_ms * 3.6` → km/h

6.5 Use Case 3 — Spark ML KMeans (Congestion Detection)

```python
from pyspark.ml.clustering import KMeans
from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(inputCols=["lat_cell", "lon_cell", "avg_aircraft"],
                            outputCol="features")
kmeans = KMeans(k=3, seed=42, featuresCol="features", predictionCol="density_cluster")
```

- Grid cells labeled **Low / Medium / High Density** by mean aircraft count per cluster
- Silhouette score evaluated with `ClusteringEvaluator`

---

7. Insights & Results

> Run date: March 12, 2026 | 67 snapshots | ~10 minutes of ingestion

7.1 Use Case 1 — Airspace Density

Metric	Value
Total aircraft records	**23,579**
Valid snapshots	**60** of 67 raw files

Metric	Value
Average aircraft / snapshot	**352.8**
Peak aircraft count	**375** (18:29 UTC)
Unique aircraft (hour 18)	**535** distinct ICAO24 codes
Average velocity	**183.7 m/s**
Average altitude	**7,527 m** (~24,700 ft)

Key finding: The airspace turned over 535 unique aircraft in one hour despite averaging only 352 at any one moment — confirming the Iberian Peninsula is a high-throughput transit corridor, not just a destination.

7.2 Use Case 2 — Country & Airline Traffic

Top countries by unique aircraft:

Country	Unique A/C	Market Share	Avg Speed (km/h)	Avg Alt (m)
Spain	**141**	**26.4%**	569	5,404
Ireland	57	10.7%	779	9,548
France	52	9.7%	602	7,076
United Kingdom	39	7.3%	765	9,685
Portugal	38	7.1%	527	5,182
Morocco	17	3.2%	**821**	**10,705**

Key findings:

- **Spain dominates at 26.4% —** driven by dense domestic short-haul operations. Spanish aircraft fly at only 5,404 m average, consistent with approach/departure phases.
- **Ireland at 10.7%** is disproportionately high for a country geographically distant from Iberia — this is Ryanair's registration country. The airline alone accounts for **83 unique aircraft**, nearly double its nearest competitor.
- **Morocco flies fastest (821 km/h) and highest (10,705 m) —** these are transatlantic overflights at cruising altitude, not Iberian domestic traffic.

- **Spain and Portugal fly 200+ km/h slower** than all other nations — clear signal of short-haul, low-altitude operations.

Top airlines:

Code	Airline	Unique A/C	Avg Speed (km/h)
RYR	Ryanair	**83**	756
VLG	Vueling	47	649
EJU	easyJet Europe	27	738
TAP	TAP Air Portugal	21	671
ANE	Air Nostrum	21	569
RAM	Royal Air Maroc	12	**858**

7.3 Use Case 3 — Congestion Detection

Metric	Value
Grid resolution	2×2 degree cells
Congestion threshold	≥5 aircraft per cell
Total cell-snapshot observations	**2,410**
Congested observations	**1,512 (62.7%)**
KMeans silhouette score	**0.4687**
High Density cells	4 cells · avg **28.5** aircraft
Medium Density cells	15 cells · avg **11.0** aircraft
Low Density cells	26 cells · avg **3.1** aircraft

Top 5 persistent congestion hotspots (congested in all 60 snapshots):

Cell	Lat	Lon	Avg Aircraft	Peak
LAT40_LON-4	40.0°N	4.0°W	**37.2**	**44**
LAT38_LON-10	38.0°N	10.0°W	26.4	29
LAT40_LON2	40.0°N	2.0°E	26.9	30
LAT36_LON-6	36.0°N	6.0°W	23.4	27
LAT38_LON-6	38.0°N	6.0°W	11.9	14

Key findings:

- **62.7% overall congestion rate —** the majority of Iberian airspace exceeds the 5-aircraft threshold at any given moment, meaning congestion is structural, not episodic.
- **LAT40_LON-4 (central Spain / Madrid FIR)** is the single most congested cell at 37.2 average aircraft and a peak of 44 — directly over one of Europe's busiest air traffic control zones.
- **LAT38_LON-10 (Atlantic entry point)** is persistently congested at 26.4 average — this cell captures transatlantic inbound/outbound traffic funneling through the Iberian gateway.
- **All 30 hotspot cells were congested in every single snapshot —** this is structural, predictable congestion that could support real-time ATC load-balancing decisions.
- **Silhouette score 0.4687** indicates meaningful but not perfect cluster separation — acceptable for geographic grid clustering where spatial autocorrelation naturally limits within-cluster variance.

---

8. Conclusions

1. **Full pipeline validated end-to-end:** NiFi → MinIO → Spark → visualisation running stably on a local OSBDET environment.
2. **Three distinct Spark capabilities exercised:** Structured Streaming, SQL, and ML — meeting all course requirements.
3. **Iberian airspace is structurally congested:** 62.7% congestion rate across all cells confirms this is baseline behaviour, not a peak-hour anomaly.
4. **Low-cost carrier dominance is visible in raw telemetry:** Ryanair alone represents ~15% of all unique aircraft detected.
5. **Two-tier altitude/speed structure:** Long-haul transit (Morocco, UK, Ireland) vs. short-haul domestic (Spain, Portugal) creates clearly separable flight profiles that KMeans captures.

9. Future Work

- **24-hour ingestion** to capture full daily traffic cycles and morning vs. evening peak comparison
- **Silver layer** Spark transformations to enrich data with nearest-airport lookup
- **Gold layer** persistence using Apache Cassandra or PostgreSQL for live dashboard serving
- **GraphFrames** (Spark Graph Processing) to model airport-to-airport routes as a directed graph — hub connectivity analysis
- **Apache Superset** dashboard connected to Gold layer for real-time operational monitoring
- **Anomaly detection** using Spark ML Isolation Forest on velocity/altitude deviations to flag unusual flight behaviour

---